

How to Build the Property Valuation Assistant — Step by Step

This is your **construction manual**. The Handbook told you *what* the project is and *why*. This document tells you *exactly how to build it* — module by module, what goes in, what comes out, and how each piece connects to the next inside one Colab notebook.

[11 modules · 1 notebook](#)[read top → bottom](#)[companion to the Handbook](#)

00

How to use this document

You are building **one Jupyter notebook in Google Colab**. Think of it as a factory line: a spreadsheet of houses enters at the top, and each station (a **module**) does one job and hands its result to the next station. A module is just **one labelled section of the notebook** — usually 1–3 code cells.

The golden rule of this project: the *output* of one module is the *input* of the next. The connection between them is always a **named Python variable**. Build the modules **in order** and keep the variable names exactly as written here, and each step will feed cleanly into the next.

How to work through it

1. Read the [Pipeline diagram](#) once, fully. That is the whole project on one screen.
2. Keep the [Data Dictionary](#) open in a second tab — it is the master list of every variable that travels between modules. It is your contract.
3. Build **in order**, **0** → **9**. Do not start Module 5 (the first model) before Module 4 has produced `x_train` and the `preprocessor`. Module 10 is optional and last.
4. For your assigned module: read its *Purpose*, copy the *Starter code* into a cell, run it, then check the "*Done when...*" box at the bottom before moving on.
5. When something breaks, check *Common errors* first — your bug is almost certainly listed there.

The starter code is a scaffold, not the finished answer. It runs, but it is intentionally minimal. Your job — and the part that earns marks — is to understand it, improve it, and explain it in the report. Copying without understanding will show immediately when you present.

One example house runs through this whole document. Meet "**the Maple St house**": a buyer describes it as *a 3-bedroom home, about 1,500 sq ft of living space, built in 2005, overall quality 7, a 2-car garage, 2 full bathrooms*. Watch it flow through to a real predicted price (≈ \$188,000) at Module 8.

01

The full pipeline — how everything connects

Every box is a module (a section of the notebook). Every **labelled arrow is the variable** that one module hands to the next. Modules 5 and 6 are **two different models**: they both read the same training data and the same `preprocessor`, and each produces its own accuracy scores, which meet again at the evaluation step (M7), where you compare them.

0 Setup
in: — → out: **libraries imported** (Groq key optional, for M9)

environment ready

1 Load data
in: **AmesHousing.csv** → out: **df_raw** (2930 × 82)

df_raw

2 Clean & pick features
in: **df_raw** → out: **df · X · y** (14 features + SalePrice)

X · y

3 Explore the data (EDA)
in: **df · y** → out: **charts** (no new variable — understanding)

X · y



02

The data dictionary — your contract

These are **every variable that travels between modules**. Spell them exactly like this in your code — that consistency is what lets each module's output feed cleanly into the next as you build the notebook in order.

VARIABLE	TYPE	CREATED IN	WHAT IT HOLDS (EXAMPLE)
<code>df_raw</code>	DataFrame	M1	The CSV exactly as loaded — 2,930 rows × 82 columns, unmodified
<code>df</code>	DataFrame	M2	The cleaned table — 2,925 rows (5 outliers dropped)
<code>NUMERIC</code>	list[str]	M2	The 13 numeric column names (<code>"Overall Qual"</code> , <code>"Gr Liv Area"</code> , ...)
<code>CATEGORICAL</code>	list[str]	M2	<code>["Neighborhood"]</code>
<code>FEATURES</code>	list[str]	M2	<code>NUMERIC + CATEGORICAL</code> — the 14 input columns
<code>TARGET</code>	str	M2	<code>"SalePrice"</code> — what we predict. Never goes into the features.
<code>X</code>	DataFrame	M2	The 14 feature columns (<code>df[FEATURES]</code>)
<code>y</code>	Series	M2	The SalePrice column (<code>df[TARGET]</code>)
<code>X_train,</code> <code>X_test</code>	DataFrame	M4	The features, split 80% / 20%
<code>y_train,</code> <code>y_test</code>	Series	M4	The prices, split the same way
<code>preprocessor</code>	ColumnTransformer	M4	The recipe that fills gaps, scales numbers, one-hot-encodes Neighborhood
<code>lr_model</code>	Pipeline	M5	Trained <i>preprocessor + Linear Regression</i>
<code>rf_model</code>	Pipeline	M6	Trained <i>preprocessor + Random Forest</i>
<code>lr_metrics,</code> <code>rf_metrics</code>	dict	M5 / M6	<code>{"rmse":..., "mae":..., "r2":...}</code> for each model

<code>eval_table</code>	DataFrame	M7	The two models' RMSE / MAE / R ² side by side
<code>winner</code>	str	M7	"Random Forest" or "Linear Regression"
<code>final_model</code>	Pipeline	M7	The chosen model (= <code>rf_model</code> or <code>lr_model</code>). The app (M8) and assistant (M9) use this one.
<code>value_house()</code>	function	M8	Prices one house from a feature dict (missing features → median)

Build in order. The evaluation (M7) cannot run until both models (M5, M6) produce their `_metrics` dicts. And neither model can train until M4 has produced `X_train` and `preprocessor`. Agree the variable names early and never change them silently.

03

Two rules that are locked — do not redesign these

These two decisions are made *for you*, because getting them wrong quietly breaks everything downstream. Everything else is yours to design.

Rule 1 — Split *before* you preprocess, and use the same split for both models

This is the single most important methodological rule in the project. Two parts:

(a) Split first, then fit cleaning only on the training set. The 20% test set must stay *completely unseen* until the very end. If you compute the median (for filling gaps) or the scaling using the whole dataset, information from the test set leaks into training — and your scores come out **falsely high**. The `Pipeline` in M5/M6 handles this automatically *because* you split in M4 first.

(b) Use the same `random_state=42` split for both models. Linear Regression and Random Forest must be judged on the *identical* train/test rows, or the comparison is meaningless (one model might just have got an easier test set). One split in M4, both models reuse it.

And never put `SalePrice` in `X`. If the price is one of the input columns, the model just reads off the answer and scores a perfect — and useless — $R^2=1.0$. `y` is the price; `X` is everything else. This is the most common silent "too good to be true" bug.

Rule 2 — The honesty guardrail: the model predicts, the LLM explains

House-price prediction is a **regression** problem on numbers. A language model cannot do that fairly — so in this project the **trained model** produces every price, and the **LLM only explains it in words** and reads the user's plain-English description.

So: the M9 prompt explicitly tells the LLM the price is *already decided* and it must **not** change or re-estimate it. Any framing like "our AI predicts house prices" is an **overclaim** and is barred from your report. The correct framing — and a point worth marks — is: *"the machine-learning model predicts; the LLM explains and provides the natural-language interface."*

04

Weekly plan — what to build, test & demo each week

This is your **sprint plan**. The weeks between Tuesday meetings are your sprints. Each sprint you **build** a set of modules, **test-run** them on the real data, and **demo them working** at the next meeting. Build strictly in order — a later module can't run until the earlier one produces its output.

A module is "done" only when you have (1) run it on the real data, (2) ticked its green *"Done when..."* box, and (3) can *show it working* at the next meeting. Code that "should work" but was never run does not count.

SPRINT	BY THIS MEETING	BUILD	TEST-RUN / DEMO AT THE MEETING
—	Mtg 1 · Kickoff · Tue 23 Jun	Meet · finalise project + roles · Week-1 plan assigned (M0–M2 starts tonight)	— (no code yet)
1	Mtg 2 · Tue 30 Jun	M0 Setup · M1 Load · M2 Clean & pick features	Notebook opens in Colab · <code>df_raw</code> loads (2930×82) · <code>x</code> is 2925×14 · <code>y</code> is SalePrice
2	Mtg 3 · Tue 7 Jul	M3 EDA · M4 Preprocess + split · M5 Linear Regression	Two EDA charts render · split sizes print · Linear Regression R^2 prints (~0.88)
3	Mtg 4 · Tue 14 Jul	M6 Random Forest · M7 Evaluate, compare & select <code>final_model</code> ★	Comparison table + charts · you can say which model won, why, and which one you committed to
4	Mtg 5 · Tue 21 Jul	M8 Predict app (single + batch) · M9 AI assistant · feature freeze · decide on optional M10	Full end-to-end run: type a house → predicted price → plain-English explanation; a batch of houses priced at once
5	Mtg 6 · Tue 28 Jul	(optional M10 UI) · clean the notebook · draft your report sections	Notebook runs top-to-bottom with no errors · each owner's draft section
—	Fri 31 Jul	—	Submit the final report to ISI

Timeline reality (read this). Kickoff (Tue 23 Jun) to deadline (31 Jul) is **≈ 5.5 working weeks across 6 meetings** — tighter than usual because Group 1 started later. That is exactly why scope is lean, why the graded core (M5–M7) lands by Meeting 4, and why you write the report *throughout*, not at the end. **No new features after the Sprint-4 freeze.** If progress is genuinely on track but time is short, your mentor may request a one-week extension from ISI — do not count on it; plan to finish by 31 Jul.

The modules, in build order

Build these in sequence. Each one: *what it does* → *exact inputs* → *exact outputs* → *how to build it* → *worked example* → *common errors* → *done-when*.

MODULE 0 Setup & environment

Sprint 1 · Mtg 2

Purpose. Import the libraries once, at the top of the notebook. Everything below depends on this cell having run. The Groq key is only needed for M9 — the whole prediction project + app (M1–M8) runs without it.

INPUT → OUTPUT

IN	OUT
nothing	all libraries importable · (optional) <code>GROQ_API_KEY</code> set for M9

STARTER CODE

```
# On Colab, run this once (remove the leading # ):
# !pip install -q scikit-learn pandas numpy matplotlib joblib groq

import os, json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# The Groq key is needed only for M9 - skip it for now.
# When you reach M9, store it as a Colab Secret (full steps + screenshot are in M9)
print("setup ok")
```

About the Groq key: you only need it for **M9** (the AI explanation) — M1–M8 run completely without it. When you reach M9, you'll store the key as a **Colab Secret** (never pasted into a code cell) using the step-by-step guide in that module.

 **For your report:** note your build environment — Google Colab, Python version, and the libraries you used (with versions: `pd.__version__` etc.). Feeds the report's *Setup / Tools* section.

✓ **Done when:** the cell runs with no red errors and prints `setup ok`.

MODULE 1 Load data

 Sprint 1 • Mtg 2

Purpose. Read the Ames housing CSV into a DataFrame and *look at it*. Do *nothing* else here — no cleaning yet. Just get the raw material into memory and confirm what you actually have.

INPUT → OUTPUT

IN	OUT
<code>AmesHousing.csv</code> (uploaded to Colab)	<code>df_raw</code> (DataFrame, 2930 × 82)

First, get the file into Colab. The CSV does not appear by itself — your mentor shares `AmesHousing.csv` on Teams. Download it, then upload it using the  **Files panel** on the left of Colab (drag the CSV in), then read it. The file disappears when the Colab session resets, so re-upload if needed (or load it from your shared Google Drive / GitHub repo).

THE DATASET — WHAT IT IS

This is **Dean De Cock's Ames, Iowa housing dataset** — **2,930 home sales, 82 columns** describing each house (size, quality, year, garage, neighbourhood...), plus the `SalePrice`. The column names contain **spaces and exact capitals** — e.g. `Gr Liv Area`, `Overall Qual`, `1st Flr SF`. Copy them exactly or pandas raises `KeyError`.

STARTER CODE

```
df_raw = pd.read_csv("AmesHousing.csv")
print("shape:", df_raw.shape)           # expect (2930, 82)
print(df_raw.columns.tolist()[:12])     # LOOK at the real column names
df_raw[["Overall Qual", "Gr Liv Area", "Year Built", "Neighborhood", "SalePrice"]].head
```

Week-1 reality check: always `print(df_raw.shape)` and look at the columns before relying on them — never assume. If `read_csv` raises `FileNotFoundError`, the file isn't uploaded to this Colab session — re-drag it into the Files panel. Report the real shape at Meeting 2.

 **For your report:** record the dataset source (Dean De Cock, Ames Iowa), its size (2,930 × 82), and the target (`SalePrice`). Feeds the *Data* chapter.

✓ **Done when:** `df_raw.shape` is `(2930, 82)` and `df_raw.head()` shows real houses with a `SalePrice` .

MODULE 2 Clean & pick the feature subset

 Sprint 1 · Mtg 2

Purpose. Two jobs. (1) Drop the 5 abnormal sales De Cock flags. (2) Choose the **14 high-signal features** instead of all 80, and split the table into `x` (inputs) and `y` (the price). Use the [feature dictionary](#) + `feature_subset.py` we gave you as your starting list.

INPUT → OUTPUT

IN	OUT
<code>df_raw</code>	<code>df</code> (clean, 2925 rows) · <code>x</code> (2925 × 14) · <code>y</code> (<code>SalePrice</code>) · the <code>NUMERIC</code> / <code>CATEGORICAL</code> / <code>FEATURES</code> lists

STARTER CODE

```
# (1) drop De Cock's 5 abnormal large houses (his own documented guidance)
before = len(df_raw)
df = df_raw[df_raw["Gr Liv Area"] <= 4000].copy()
print(f"dropped {before - len(df)} outliers; rows now {len(df)}")

# (2) the 14-feature subset (copy from feature_subset.py)
NUMERIC = ["Overall Qual", "Gr Liv Area", "Total Bsmt SF", "Garage Cars", "Garage Area",
           "Year Built", "Year Remod/Add", "Full Bath", "TotRms AbvGrd", "1st Flr SF",
           "Lot Area", "Fireplaces", "Bedroom AbvGr"]
CATEGORICAL = ["Neighborhood"]
TARGET = "SalePrice"
FEATURES = NUMERIC + CATEGORICAL

X = df[FEATURES].copy()
y = df[TARGET].copy()
print("X:", X.shape, " y:", y.shape)
```

Why drop the 5 houses? De Cock (who built this dataset) documents 5 sales with `Gr Liv Area` over 4000 sq ft that sold for far less than their size implies (partial sales / unusual deals). They are genuine oddities that distort a model, so removing them is standard practice for this dataset — and a good honest line for your report.

⚠ **Column names are literal.** "Gr Liv Area" has two spaces and that capitalisation; "1st Flr SF" starts with a digit; "Year Remod/Add" has a slash. Copy them *exactly* from `feature_subset.py` — a typo gives `KeyError: 'Gr liv area'`.

WORKED EXAMPLE — THE REAL NUMBERS

On the real file this produces: `df_raw` = 2,930 rows → drop 5 → `df` = 2,925 rows. `x` is 2,925 × 14; `y` is the 2,925 sale prices. (A handful of cells in `Total Bsmt SF`, `Garage Cars`, `Garage Area` are blank — that's fine, M4 fills them.)

📄 **For your report:** document the cleaning (5 outliers dropped, why) and list the 14 features you chose *and why* (the feature dictionary's correlations are good evidence). Note that `SalePrice` is the target, never an input. Feeds the *Data* chapter.

✅ **Done when:** `X.shape` is `(2925, 14)`, `y.shape` is `(2925,)`, and `"SalePrice"` is **not** in `X.columns`.

MODULE 3 Explore the data (EDA)

📅 Sprint 2 · Mtg 3

Purpose. Before modelling, *look* at the data. Two charts tell the story: how prices are spread, and which features move most tightly with price. This is where you build intuition for what the model will later confirm.

INPUT → OUTPUT

IN	OUT
<code>df</code> · <code>y</code>	two charts + a printed correlation table (understanding — no new pipeline variable)

STARTER CODE

```
# chart 1 - how SalePrice is distributed (you'll see it's right-skewed)
plt.figure(figsize=(6,4))
plt.hist(y, bins=40, color="#4C72B0", edgecolor="white")
plt.title("SalePrice distribution"); plt.xlabel("SalePrice ($)"); plt.ylabel("count")
plt.tight_layout(); plt.show()

# chart 2 - which numeric features correlate most with price
corr = df[NUMERIC + [TARGET]].corr()[TARGET].drop(TARGET).sort_values()
plt.figure(figsize=(6,5))
plt.barh(corr.index, corr.values, color="#55A868")
plt.title("Correlation of each feature with SalePrice"); plt.xlabel("Pearson r")
plt.tight_layout(); plt.show()
print(corr.sort_values(ascending=False).round(2))
```

What "correlation" means here, plainly: a number from -1 to +1 saying how tightly a feature moves with price. Overall Qual ($\approx +0.81$) and Gr Liv Area ($\approx +0.72$) lead — bigger, higher-quality houses cost more. Bedroom AbvGr is surprisingly weak ($\approx +0.14$): once you know a house's size and quality, the raw bedroom count adds little. That contrast is a great report observation.

 **For your report:** include both charts and 2–3 sentences interpreting them (the skew in prices; the strongest and weakest correlates, and why the weak ones surprised you). Feeds the *Data/EDA* chapter.

 **Done when:** both charts render, and your printed correlation list has Overall Qual at the top.

MODULE 4 Preprocess + split (the steep one — read slowly)

 Sprint 2 · Mtg 3

Purpose. Models need clean numbers, and an honest test. So we (a) **split** the data into train/test *first*, then (b) build a `preprocessor` recipe that fills blanks, scales numbers, and turns the `Neighborhood` text into 0/1 columns. We *define* the recipe here; each model (M5, M6) *applies* it inside its own pipeline. **This module is given to you as a scaffold — you are not expected to invent it. Understand it and explain it.**

INPUT → OUTPUT

IN	OUT
<code>X</code> · <code>y</code>	<code>X_train, X_test, y_train, y_test</code> · <code>preprocessor</code>

STARTER CODE

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)  # SAME seed for both models = fair
print(f"train={len(X_train)} test={len(X_test)}")

# recipe for the 13 numeric columns: fill blanks with the median, then scale
numeric_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
])

# recipe for Neighborhood: fill blanks, then one-hot encode (text -> 0/1 columns)
categorical_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])

# bundle both recipes, each pointed at its columns
preprocessor = ColumnTransformer([
    ("num", numeric_pipe, NUMERIC),
    ("cat", categorical_pipe, CATEGORICAL),
])

print("preprocessor ready")
```

Reading this, piece by piece (it looks heavy, but it is four simple ideas):

- `train_test_split(... test_size=0.2, random_state=42)` — randomly hold back **20% of houses as a test set** the model never trains on. `random_state=42` just fixes the shuffle so you get the same split every run (and so both models get the *same* split — Rule 1).
- `SimpleImputer(strategy="median")` — some cells are blank; **fill each blank with that column's median** value.
- `StandardScaler()` — put every numeric column on the **same scale** (so `Lot Area` in the thousands doesn't dwarf `Overall Qual` from 1–10). Linear Regression needs this; it doesn't hurt the forest.
- `OneHotEncoder()` — a model can't do maths on the word `"NAmes"`, so turn `Neighborhood` into one 0/1 column per neighbourhood. `handle_unknown="ignore"` means a neighbourhood not seen in training won't crash it.
- `ColumnTransformer` — applies the numeric recipe to the 13 number columns and the categorical recipe to `Neighborhood`, in one object.

Why split BEFORE building the recipe? Because the median and the scaling must be learned from the *training data only*. If they peek at the test set, your scores are fake (Rule 1a). You don't call `.fit()` here at all — the model's `Pipeline` in M5/M6 will fit the preprocessor on `X_train` for you, automatically and correctly.

 **For your report:** explain, in plain words, the train/test split (and why the test set must stay unseen), median-imputation, scaling, and one-hot encoding. This is your *Methodology – data preparation* section. A clear explanation here earns marks.

✅ **Done when:** it prints `train=2340 test=585` and `preprocessor ready` with no error.

MODULE 5 Model A — Linear Regression (the baseline)

📅 Sprint 2 · Mtg 3

Purpose. The simplest honest model: fit a straight-line relationship between the features and the price. It is your **baseline** — the yardstick the fancier model must beat. We wrap `preprocessor + LinearRegression` in one `Pipeline` so the cleaning is applied automatically and correctly.

NEW CONCEPT — WHAT THE THREE SCORES MEAN (READ FIRST)

METRIC	IN ONE SENTENCE	BETTER IS...
RMSE	typical error in dollars, with big misses punished extra	lower
MAE	average error in dollars, plain and simple	lower
R²	share of price variation the model explains, 0→1 (1 = perfect)	higher

INPUT → OUTPUT

IN	OUT
<code>X_train, y_train, X_test, y_test</code> · <code>preprocessor</code>	<code>lr_model</code> (trained) · <code>lr_metrics</code> (dict)

STARTER CODE

```
def rmse(y_true, y_pred):
    return float(np.sqrt(mean_squared_error(y_true, y_pred)))

lr_model = Pipeline([("prep", preprocessor), ("model", LinearRegression())])
lr_model.fit(X_train, y_train) # fits preprocessor + model together
lr_pred = lr_model.predict(X_test)

lr_metrics = {"rmse": rmse(y_test, lr_pred),
              "mae": float(mean_absolute_error(y_test, lr_pred)),
              "r2": float(r2_score(y_test, lr_pred))}
print(f"Linear Regression RMSE=${lr_metrics['rmse']:.0f} "
      f"MAE=${lr_metrics['mae']:.0f} R2={lr_metrics['r2']:.3f}")
```

The one idea that makes this clean: `Pipeline([("prep", preprocessor), ("model", LinearRegression())])` glues the cleaning recipe to the model. When you call `.fit(X_train, y_train)`, it fits the preprocessor on the *training* rows and then trains the model — so the test set is never touched. That is Rule 1, handled for you.

Common error: `NameError: preprocessor is not defined` → you haven't run M4.
`ValueError: Input contains NaN` → you're feeding the model raw `X` instead of going through the pipeline (the pipeline is what fills the blanks).

 **For your report:** explain Linear Regression in plain English, define RMSE/MAE/R², and report this baseline's three numbers. Feeds *Methodology (baseline)*.

✓ **Done when:** it prints three numbers and the R² is around **0.877**.

MODULE 6 Model B — Random Forest (the smarter one)

 Sprint 3 · Mtg 4

Purpose. The same task — predict price — but with a model that captures curves and interactions a straight line can't. A **Random Forest** grows hundreds of decision trees on random slices of the data and averages them. The pipeline shape is *identical* to M5 — only the model line changes.

NEW CONCEPT — A RANDOM FOREST IN ONE PICTURE

A single **decision tree** asks yes/no questions ("is Overall Qual ≥ 7 ? is Gr Liv Area ≥ 1800 ?) until it lands on a price. One tree alone overfits. A **forest** builds many trees, each on a random subset of rows and features, and averages their answers — so the quirks cancel out and the signal remains. You don't scale for it (trees don't care about scale), but leaving scaling in does no harm, so we keep the same preprocessor.

INPUT → OUTPUT

IN	OUT
<code>X_train, y_train, X_test, y_test</code> · <code>preprocessor</code>	<code>rf_model</code> (trained) · <code>rf_metrics</code> (dict)

STARTER CODE

```
rf_model = Pipeline([
    ("prep", preprocessor),
    ("model", RandomForestRegressor(n_estimators=300, random_state=42, n_jobs=-1)),
])
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)

rf_metrics = {"rmse": rmse(y_test, rf_pred),
              "mae": float(mean_absolute_error(y_test, rf_pred)),
              "r2": float(r2_score(y_test, rf_pred))}
print(f"Random Forest      RMSE=${rf_metrics['rmse']:.0f}  "
      f"MAE=${rf_metrics['mae']:.0f}  R2={rf_metrics['r2']:.3f}")
```

The settings, decoded: `n_estimators=300` = grow 300 trees (more = steadier, slower). `random_state=42` = reproducible. `n_jobs=-1` = use all CPU cores so it trains faster. Reuse the *same* `rmse` helper and the *same* train/test split from M4/M5 — that's what makes the comparison fair.

First run is a little slow (300 trees on 2,340 houses) — a few seconds, normal. If R^2 comes out *worse* than Linear Regression, something is off (usually a different/re-done split) — both models must use the M4 split.

 **For your report:** explain a decision tree → a forest in your own words, and report this model's three numbers next to the baseline. Feeds *Methodology (Random Forest)*.

✓ **Done when:** it prints three numbers and the R^2 (≈ 0.919) is *higher* than Linear Regression's.

MODULE 7 Evaluate, compare & select the final model ★

 Sprint 3 • Mtg 4

Purpose. This is the graded heart of the project. Put both models side by side, **decide which one wins** by R^2 , and **commit to it** as `final_model` — the single model the app (M8) and the assistant (M9) will use. Then read **which features the Random Forest leaned on**. Outputs: a comparison table, the decision, two charts.

IN	OUT
lr_metrics · rf_metrics · lr_model · rf_model · y_test · rf_pred	eval_table · winner · final_model · two charts

STARTER CODE

```
eval_table = pd.DataFrame({
    "model": ["Linear Regression", "Random Forest"],
    "RMSE": [lr_metrics["rmse"], rf_metrics["rmse"]],
    "MAE": [lr_metrics["mae"], rf_metrics["mae"]],
    "R2": [lr_metrics["r2"], rf_metrics["r2"]],
})
winner = "Random Forest" if rf_metrics["r2"] > lr_metrics["r2"] else "Linear Regres
print(eval_table.round(3).to_string(index=False)); print("winner:", winner)

# THE DECISION: commit to the winner. M8 (app) and M9 (LLM) use final_model.
final_model = rf_model if winner == "Random Forest" else lr_model
print("final_model ->", winner)

# which features mattered (read straight from the trained forest)
feat_names = rf_model.named_steps["prep"].get_feature_names_out()
importances = rf_model.named_steps["model"].feature_importances_
top = pd.Series(importances, index=feat_names).sort_values(ascending=False).head(10)

plt.figure(figsize=(6,5))
plt.barh(top.index[::-1], top.values[::-1], color="#C44E52")
plt.title("Random Forest - top-10 feature importances"); plt.xlabel("importance")
plt.tight_layout(); plt.show()

plt.figure(figsize=(5,5)) # actual vs predicted: tight = g
plt.scatter(y_test, rf_pred, alpha=0.4, color="#4C72B0", edgecolor="none")
lims = [y_test.min(), y_test.max()]; plt.plot(lims, lims, "k--")
plt.title("Random Forest - actual vs predicted"); plt.xlabel("actual"); plt.ylabel(
plt.tight_layout(); plt.show()
```

The dense line, decoded: `rf_model.named_steps["model"].feature_importances_` — a trained forest records **how much each feature reduced its prediction error**; that's the "importance". `get_feature_names_out()` gives the matching names (one-hot makes many Neighborhood_* columns, so names are prefixed num__ / cat__).
`.sort_values(...).head(10)` keeps the top 10. The **actual-vs-predicted** scatter: points hugging the dashed diagonal = accurate predictions.

Verified result to expect: Random Forest wins — R^2 **0.919** vs **0.877**; RMSE **\$24,011** vs **\$29,581**. Top driver by far: Overall Qual (≈ 0.62), then Gr Liv Area (≈ 0.14). If your numbers are wildly different, check you used the M4 split for both models.

Go further for more marks: the headline single number can flatter a model. Dig deeper — plot the *residuals* (errors) to see where the model struggles (it tends to under-predict the most expensive homes), report error in *percentage* terms, or test how the ranking changes if you add more features. *This is where your ambition belongs.*

 **For your report — this is your *Results* chapter:** paste `eval_table`, both charts, and 3–4 sentences on *which model won, by how much, and why* (a forest captures non-linear effects a line can't), then state *which model you committed to and why*. Discuss what the importance chart says about the housing market (quality + size dominate). This section carries the most marks — invest here.

✓ **Done when:** `eval_table` shows both models' three scores, both charts render, `winner` holds "Random Forest", and `final_model` is set to the winning pipeline.

MODULE 8

Predict prices with your model — the valuation app (single + batch)

 Sprint 4 ·
Mtg 5

Purpose — this is the payoff. Everything so far *built and chose* a model. Now you **use** it: feed it a house it has never seen and get a price. That is the whole point of the project — a buyer, seller, or bank could type in a house's features and get an instant estimate. Two modes: **(1) single house** — specify a few features, get one price; **(2) batch** — hand it a whole table of houses, get a price for every row.

Why this module matters most for *understanding*: a model you only score with R^2 is an abstraction. A model you can *poke* — change Overall Qual from 6 to 8 and watch the price jump — is where you actually feel what it learned. Play with it.

INPUT → OUTPUT

IN	OUT
<code>final_model</code> · a house (a feature dict) or a table of houses	a predicted price (single) · a price column on the table (batch)

STARTER CODE

```
# sensible fallbacks for any feature the user doesn't specify
defaults = {c: float(X[c].median()) for c in NUMERIC}
defaults["Neighborhood"] = X["Neighborhood"].mode().iloc[0]
neighborhoods = sorted(X["Neighborhood"].unique())

def value_house(features):
    # features overrides defaults; anything not given falls back to the median
    row = {**defaults, **{k: v for k, v in features.items() if k in FEATURES}}
    return float(final_model.predict(pd.DataFrame([row])[FEATURES])[0])

# MODE 1 - single house: edit these and re-run to play around
my_house = {"Overall Qual": 7, "Gr Liv Area": 1500, "Year Built": 2005,
            "Garage Cars": 2, "Full Bath": 2, "Bedroom AbvGr": 3}
print(f"single-house estimate: ${value_house(my_house):,.0f}") # -> about $188,000

# MODE 2 - batch: a whole table of houses at once
batch = X_test.head(5).copy() # or build your own DataFrame of house
batch["predicted_price"] = final_model.predict(batch)
print(batch[["Overall Qual", "Gr Liv Area", "Neighborhood", "predicted_price"]].to_str
```

How `value_house()` works: you pass only the features you care about (say quality + size); it fills every other feature with that column's median, builds a one-row DataFrame in the exact `FEATURES` order, and asks `final_model` for a price. Batch mode is even simpler — a Pipeline can `.predict()` a whole table at once, so you just attach the result as a new column.

Common error: `KeyError` or a column-order crash → your house dict / table must use the *exact* feature names (and batch tables must contain all 14 `FEATURES` columns). Reuse `defaults` + `FEATURES` and it lines up.

 **For your report:** show one single-house prediction and one batch table, and write 2–3 sentences on the *use case* — who would use this and why an instant estimate is valuable. This is your “*utility of the model*” discussion. Feeds *Results / Discussion*.

✓ **Done when:** `value_house(my_house)` returns a sensible price (≈ \$188,000 for the example), and the batch table shows a `predicted_price` for every row.

Purpose. Put a friendly AI layer on top of the predictions from M8. Two features: (1) the model has priced a house, and the **LLM explains *why* in plain English** (works for a single house or each house in a batch); (2) a user **describes a house in words**, the LLM turns that into features, and the **model** prices it via `value_house()`. Remember [Rule 2](#): the model predicts, the LLM only explains and interfaces.

INPUT → OUTPUT

IN	OUT
<code>final_model</code> · a prediction from M8 (single or batch) or free text · <code>GROQ_API_KEY</code>	a plain-English explanation · a price for a described house

FIRST: GET YOUR GROQ KEY AND STORE IT AS A COLAB SECRET

This module needs a free Groq API key — and the safe place to keep it is Colab's  **Secrets** panel, *not* typed into a code cell (a key typed in a cell gets pushed to GitHub and disabled). Two steps:

Part A — Create your free Groq key (no credit card):

1. Go to <https://console.groq.com> and sign up for a free account.
2. In the left sidebar, click **API Keys**.
3. Click **Create API Key**, give it a name (e.g. `ISI Property Valuation`), and create it.
4. **Copy the key immediately** — it starts with `gsk_` and is shown *only once*. If you lose it, just create another.

Part B — Add it as a Colab Secret named `GROQ_API_KEY` :

1. In your Colab notebook, click the  key icon in the far-left sidebar to open **Secrets**.
2. Click **+ Add new secret**.
3. In **Name**, type `GROQ_API_KEY` exactly (all caps, with underscores — the code looks for this exact name).
4. In **Value**, paste your `gsk_...` key.
5. Turn the **Notebook access** toggle (left of the row) **ON** — without this, your notebook cannot read the secret.



The `GROQ_API_KEY` row in Colab's  Secrets panel. **Note this crop is only half-done** — to finish, you must **paste your `gsk_...` value** into the empty Value box *and* switch the **Notebook access toggle (far left) ON**. It only works once *both* are set.

STARTER CODE

```
# load your Groq key from the Colab Secret into the environment
from google.colab import userdata
os.environ["GROQ_API_KEY"] = userdata.get("GROQ_API_KEY")

from groq import Groq
client = Groq() # automatically reads GROQ_API_KEY from the environment
GROQ_MODEL = "llama-3.3-70b-versatile" # verify current models at console.groq.co

# the model's top drivers, cleaned of the num__/cat__ prefixes (from M7's `top`)
drivers = [n.replace("num__", "").replace("cat__", "") for n in top.index[:4]]

def explain_valuation(house, predicted_price, drivers):
    facts = "\n".join(f"- {k}: {v}" for k, v in house.items())
    prompt = (
        "You are a property-valuation assistant. A machine-learning model has ALREA"
        f"estimated this house's price at ${predicted_price:,.0f}. Do NOT change or"
        "re-estimate the price. In 3-4 sentences, explain to a buyer why it lands n"
        f"that figure, using its attributes and the model's main drivers "
        f"({'', '.join(drivers)}).\n\nAttributes:\n{n{facts}}")
    resp = client.chat.completions.create(
        model=GROQ_MODEL, messages=[{"role": "user", "content": prompt}], temperature=
    return resp.choices[0].message.content.strip()

# DEMO - explain the single house you priced in M8
sample = my_house
sample_price = value_house(my_house)
print(explain_valuation(sample, sample_price, drivers))

# BATCH - explain each row of the M8 batch table (loop over predictions)
for _, h in batch.iterrows():
    expl = explain_valuation(h[FEATURES].to_dict(), h["predicted_price"], drivers)
    print(f"${h['predicted_price']:,.0f}:", expl[:120], "...")
```

The honesty guardrail, in code. Notice the prompt says *"a model has ALREADY estimated... Do NOT change or re-estimate the price."* That single instruction is what keeps the LLM in its lane — explaining, not guessing prices. Keep it. In your report, state clearly that the model predicts and the LLM explains.

```
def parse_house_text(text, allowed_neighborhoods):
    prompt = (
        "Extract house attributes from the user's text. Return ONLY JSON with the k
        \"you can infer (omit unknown): Overall Qual, Gr Liv Area, Garage Cars, Year
        f\"Full Bath, Bedroom AbvGr, Neighborhood (one of: {'', '}.join(allowed_neighb
        f\"User: {text}\"))
    resp = client.chat.completions.create(
        model=GROQ_MODEL, messages=[{"role": "user", "content": prompt}],
        temperature=0, response_format={"type": "json_object"})
    return json.loads(resp.choices[0].message.content)

# reuse M8's value_house() + neighborhoods - free text -> features -> price
text = "A 3-bedroom house, ~1500 sq ft, built 2005, overall quality 7, 2-car gara
parsed = parse_house_text(text, neighborhoods)
price = value_house(parsed) # the Maple St house -> about $188
print("predicted:", f"${price:,.0f}")
print(explain_valuation(parsed, price, drivers))
```

Auth error / `GroqError: api_key must be set` / `KeyError: 'GROQ_API_KEY'` ? Two common causes: (1) the secret must be named exactly `GROQ_API_KEY` — not `GROQ`; (2) a Colab Secret does *not* appear in `os.environ` on its own — you must pull it in with `userdata.get("GROQ_API_KEY")` as shown above (`os.environ.get(...)` alone returns `None`). Re-run this cell after setting the secret. **"model not found"**? The model name was retired; check current free models at console.groq.com/docs/models and swap it in. Do not guess names.

The assistant is an add-on, not load-bearing. If the Groq call fails on demo day, your prediction study + app (M1–M8) still stands on its own and still earns the marks. That is by design.

 **For your report:** include one real explanation and one describe-a-house example (with its predicted price), and 2–3 sentences on what the LLM adds and its limits — and restate the honesty guardrail (model predicts, LLM explains). Feeds the *AI assistant / Conclusion* section.

 **Done when:** the LLM prints a 3–4 sentence explanation that does *not* state a different price, and (if attempted) the described house gets a sensible predicted price.

Purpose. Wrap Modules 8→9 behind simple input boxes so a non-coder can try it — type a house, see a price and an explanation. **Do not start this until the notebook is complete and the report is drafted.** It earns no extra marks on its own — it is a demo nicety on top of the app you already built in M8.

Simplest path: `ipywidgets` inside the notebook — a text box for the house description (or sliders for the key features) and a button that runs the model + the explainer. A `streamlit` app is also fine but is more work and optional.

SKETCH — INSIDE THE NOTEBOOK

```
import ipywidgets as widgets
desc = widgets.Textarea(description="Describe the house:")
hood = widgets.Dropdown(options=neighborhoods, description="Neighborhood:")
go = widgets.Button(description="Value it")
display(desc, hood, go)
# on click: parse_house_text(desc.value) -> model.predict -> explain_valuation
```

Reuse Rule 2's spirit: the UI must still let the *model* produce the price — the text box only describes the house. Never present an LLM-typed number as the price.

 **For your report:** if you built it, add a screenshot and a line on how it works. If you didn't, list it as *future work* in the Conclusion — a perfectly good outcome.

✓ **Done when:** (only if attempted) entering a description + clicking the button prints a predicted price and an explanation.

06

Where this goes next (extensions — after the project)

What you built is not a one-off. The pipeline — *load*→*pick features*→*split & clean*→*train models*→*compare & pick a winner*→*predict*→*explain* — is a **reusable template**. Two directions worth exploring once the project is submitted:

1 · Same problem, your own data — an Indian housing dataset

Nothing here is specific to Ames, Iowa. To value **Bengaluru or other Indian homes**, swap in an Indian housing CSV (several are on Kaggle), point M1 at it, and adjust the feature names in M2 to that file's columns. *Everything from M3 onward is unchanged.*

Honest heads-up: Indian housing data is usually messier than the clean Ames file — more missing values, prices in lakhs/crores, locality as free text. That's exactly why we learned on Ames first: get the method solid on clean data, then take it somewhere harder. A great post-project challenge, not part of the graded scope.

2 · Same recipe, completely different problems

The bigger idea: this *workflow* predicts far more than house prices. Anywhere you have past examples with an outcome, you can train, compare, and use a model the same way:

PROBLEM	PREDICT...	WHAT CHANGES
Customer churn	will this customer leave? (yes/no)	a <i>classification</i> task — swap <code>RandomForestRegressor</code> → <code>RandomForestClassifier</code> , and R^2 /RMSE → accuracy / precision / recall
Employee attrition	will this employee resign? (yes/no)	same classification swap; features become tenure, salary band, role, etc.
Loan default	will this loan go bad? (yes/no)	same; plus you'd care a lot about precision/recall, not just accuracy
Demand / sales	how many units next month? (a number)	still <i>regression</i> — almost identical to this project

The one honest distinction: house price is a *number*, so this is **regression**. Churn / attrition / default are *yes-or-no*, so they're **classification** — you change the model family and the metrics, but the *shape* of the pipeline (load → features → split → train → compare → predict → explain) is identical. Learn it once here; reuse it everywhere. A strong line for your report's *Future Work*.

Plain-English glossary

TERM	IN ONE SENTENCE
Regression	Predicting a number (here, a price) from input features.
Classification	Predicting a category instead of a number (e.g. will-churn yes/no). Same workflow, different model family + metrics — see §06.
Final model	The single model you commit to after comparing (M7) — the one your app uses.
Feature / target	A feature is an input column (size, quality...); the target is what you predict (<code>SalePrice</code>).
Train / test split	Holding back some rows the model never trains on, so you can measure how it does on unseen houses.
Data leakage	Letting test-set (or answer) information sneak into training — it makes scores look great but fake.
Imputation	Filling in missing values (here, with the column median).
Scaling	Putting numeric columns on a comparable scale so big-numbered features don't dominate.
One-hot encoding	Turning a text category (Neighborhood) into a set of 0/1 columns a model can use.
Linear Regression	The baseline model: fits a straight-line relationship between features and price.
Random Forest	Many decision trees averaged together — captures curves and interactions a line can't.
RMSE / MAE	Average prediction error in dollars (RMSE punishes big misses more). Lower is better.
R²	Share of price variation the model explains, 0→1. Higher is better.
Feature importance	How much each feature helped the forest reduce its error — which inputs mattered most.

